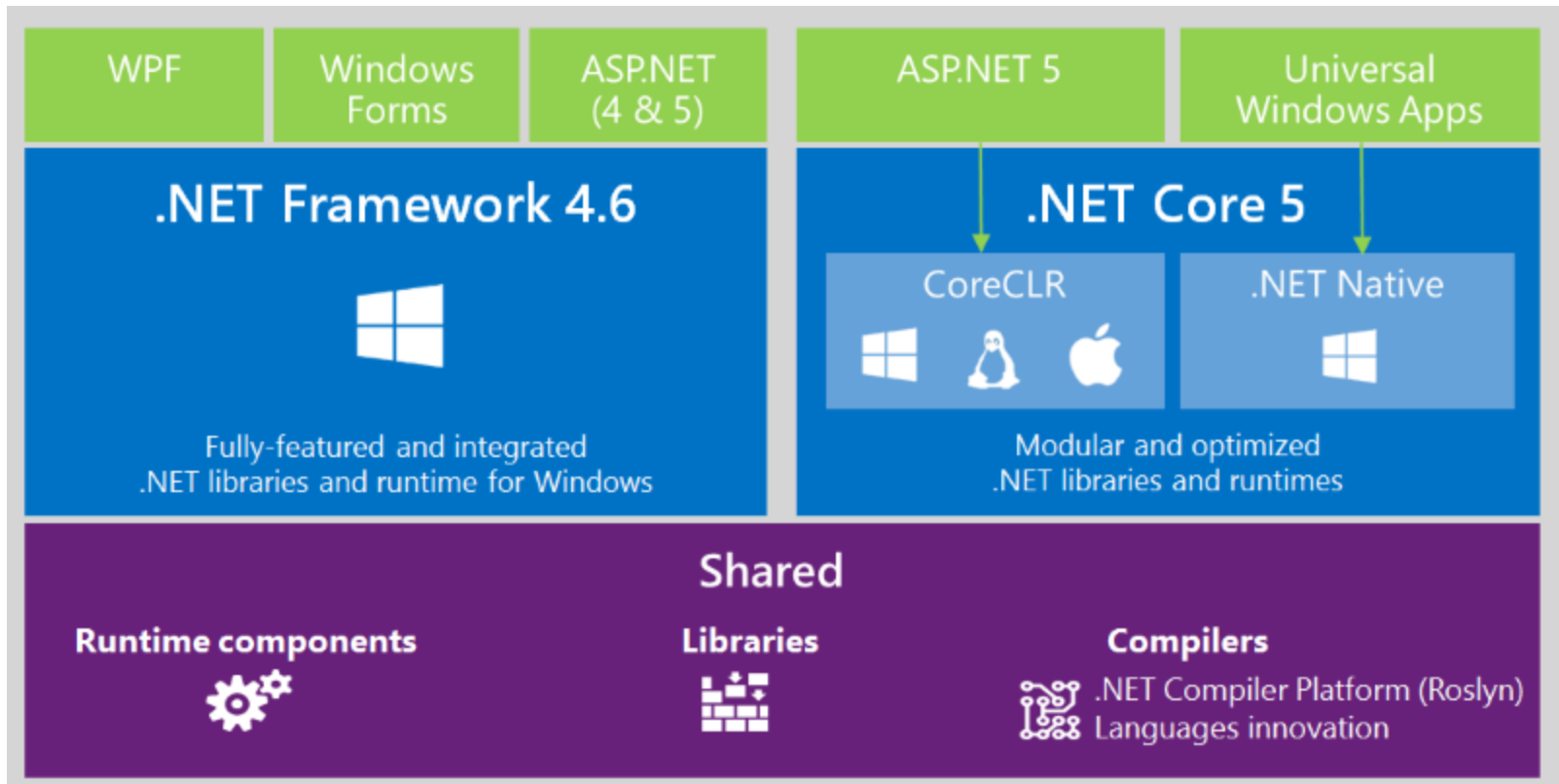


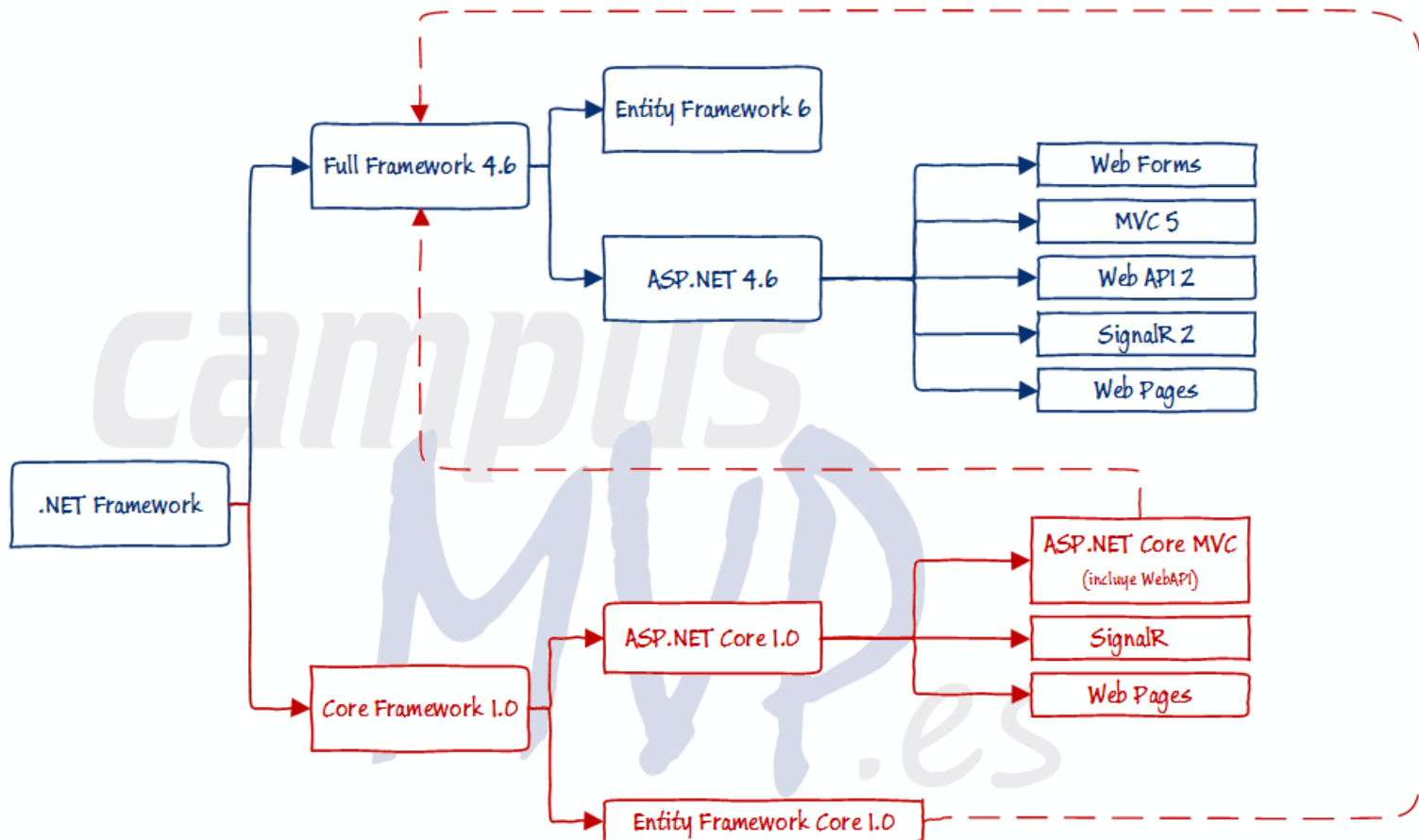
Arquitectura de Software

Introducción a la Plataforma NET

.NET Framework



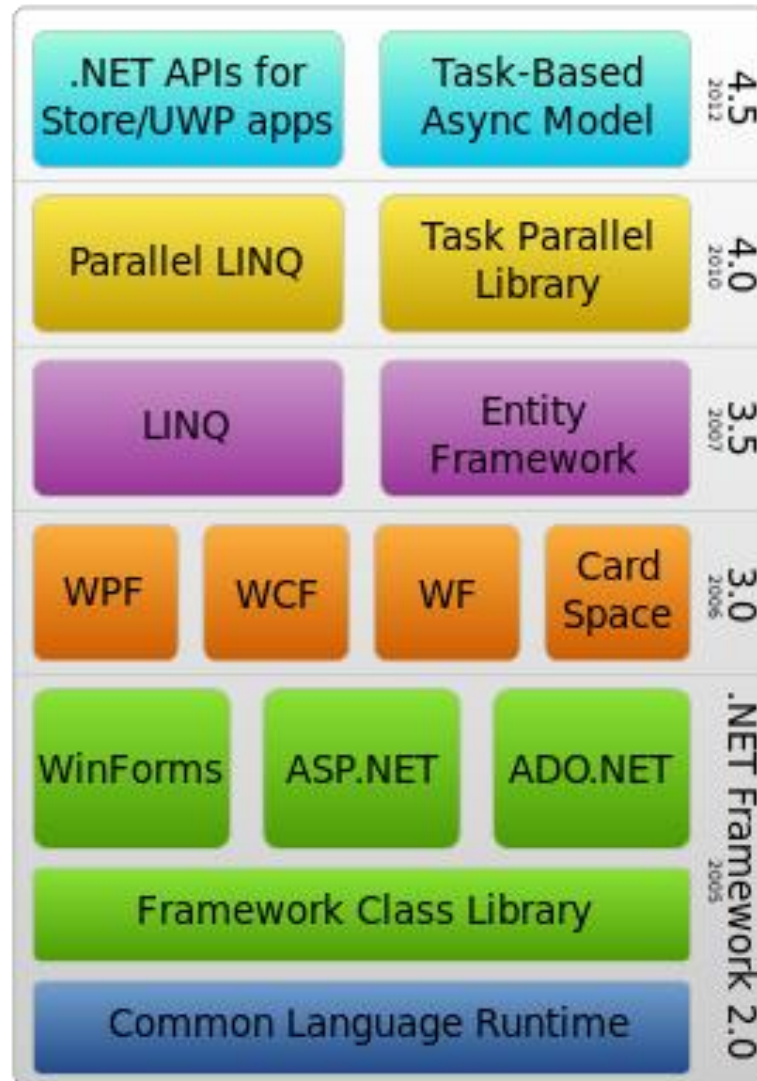
.NET Framework



.NET Framework

- .NET Full Framework
 - solo para Windows
- .NET Core
 - multiplataforma y totalmente Open Source

.NET Framework 4.5



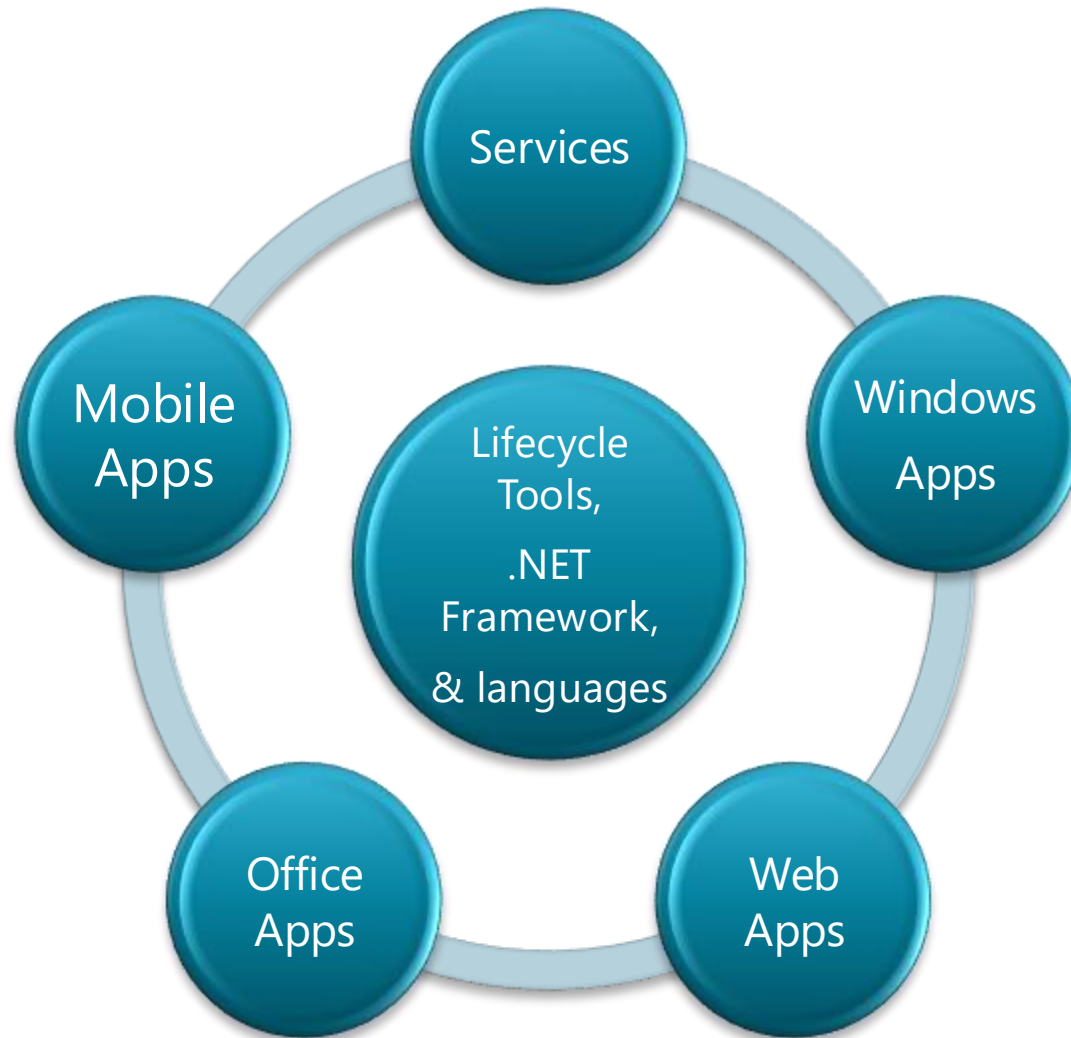
Common Language Runtime (CLR)

- Lenguaje Común en tiempo de Ejecución
- Es la implementación del estándar Common Language Infrastructure (CLI) que define un ambiente de ejecución para los codigos de los programas.
- El CLR ejecuta una forma de código intermedio (bytecode) llamada Common Intermediate Language (CIL), la implementación de Microsoft del CLI.

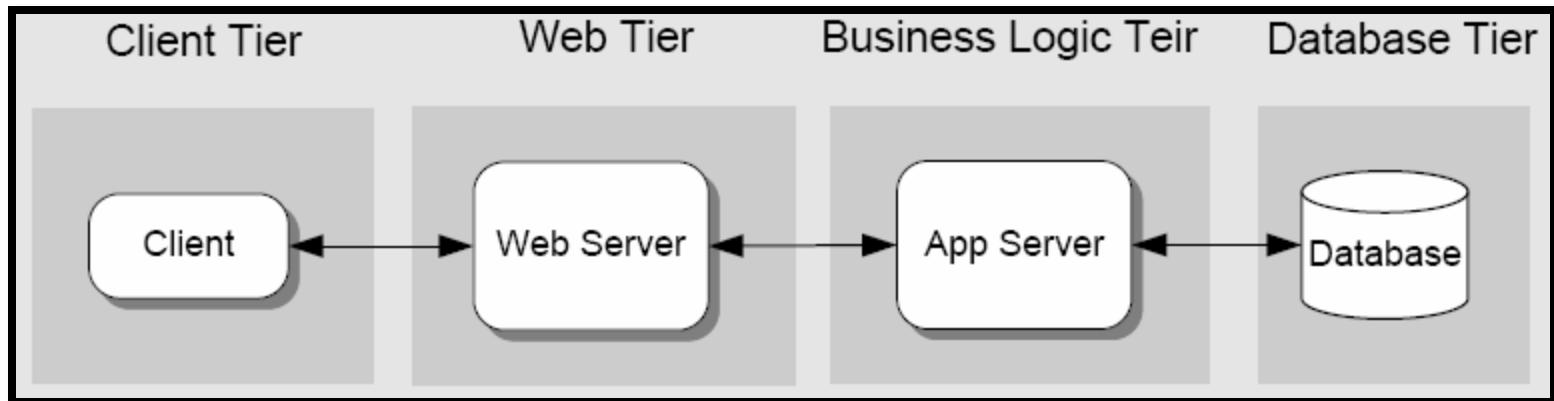
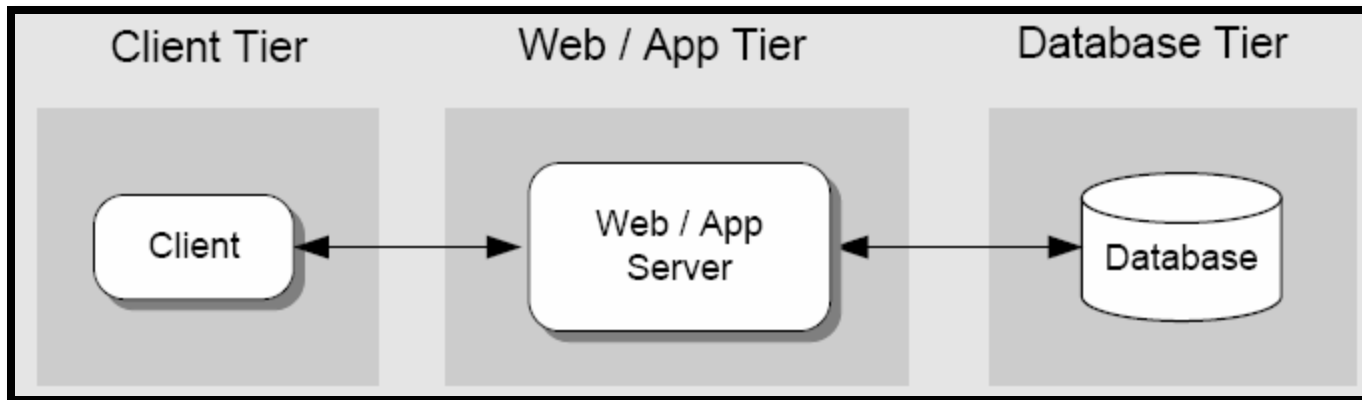
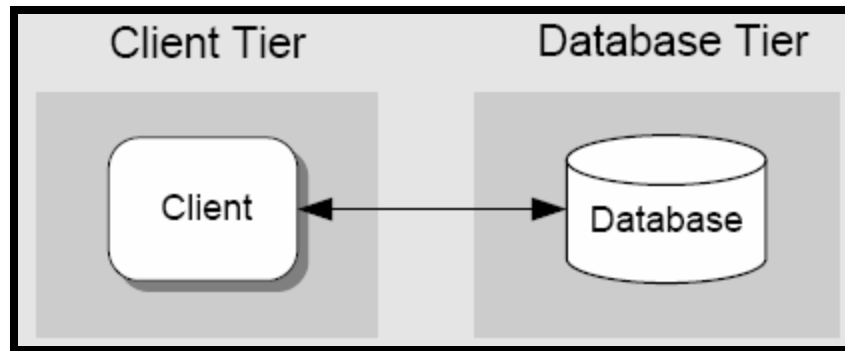
Common Language Runtime (CLR)

- Las aplicaciones que se ejecutan dentro del CLR se denominan *managed applications*, y la mayoría de las aplicaciones .NET son desarrolladas usando managed code (código que se ejecutará dentro del CLR).
- El CLR también provee servicios como seguridad, manejo de memoria, y manejo de excepciones.

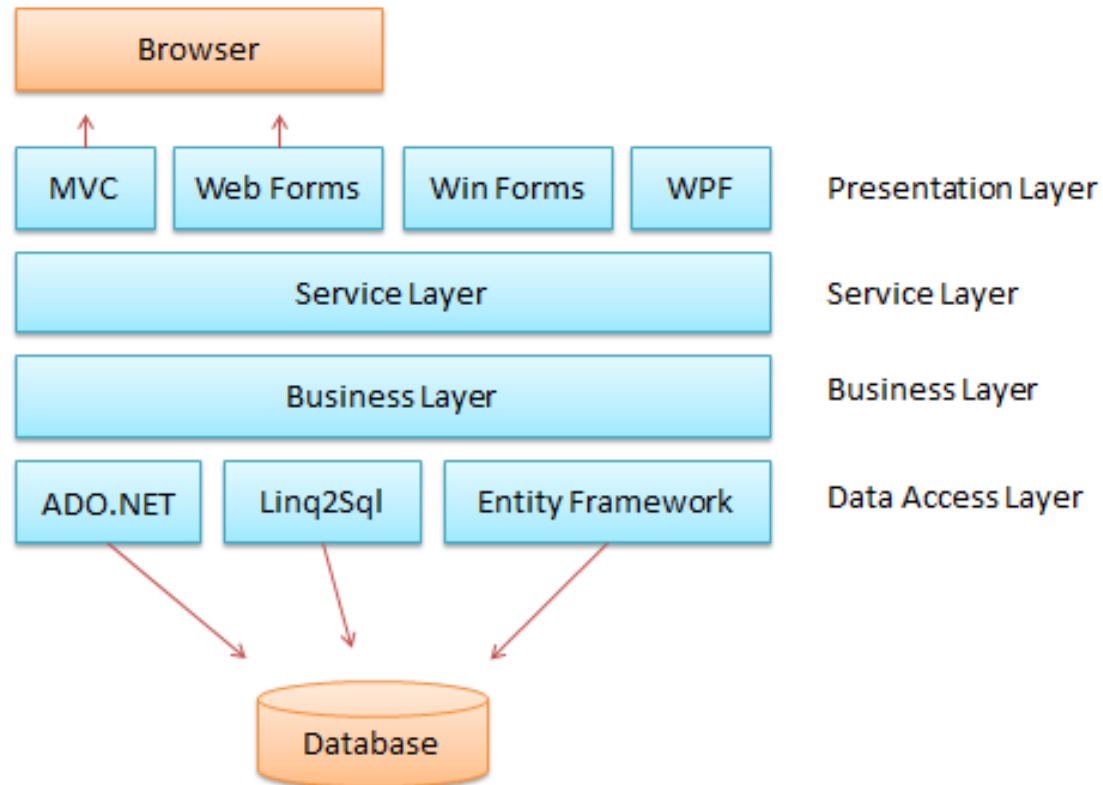
Aplicaciones .NET



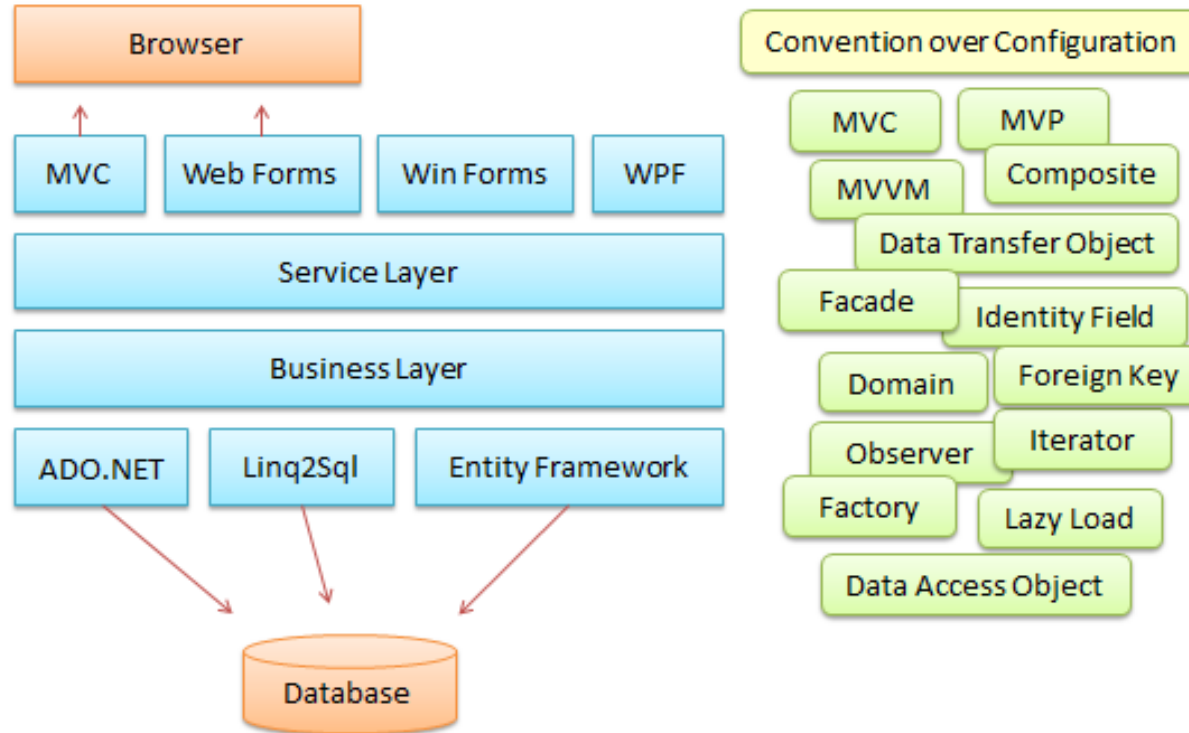
n- Tier



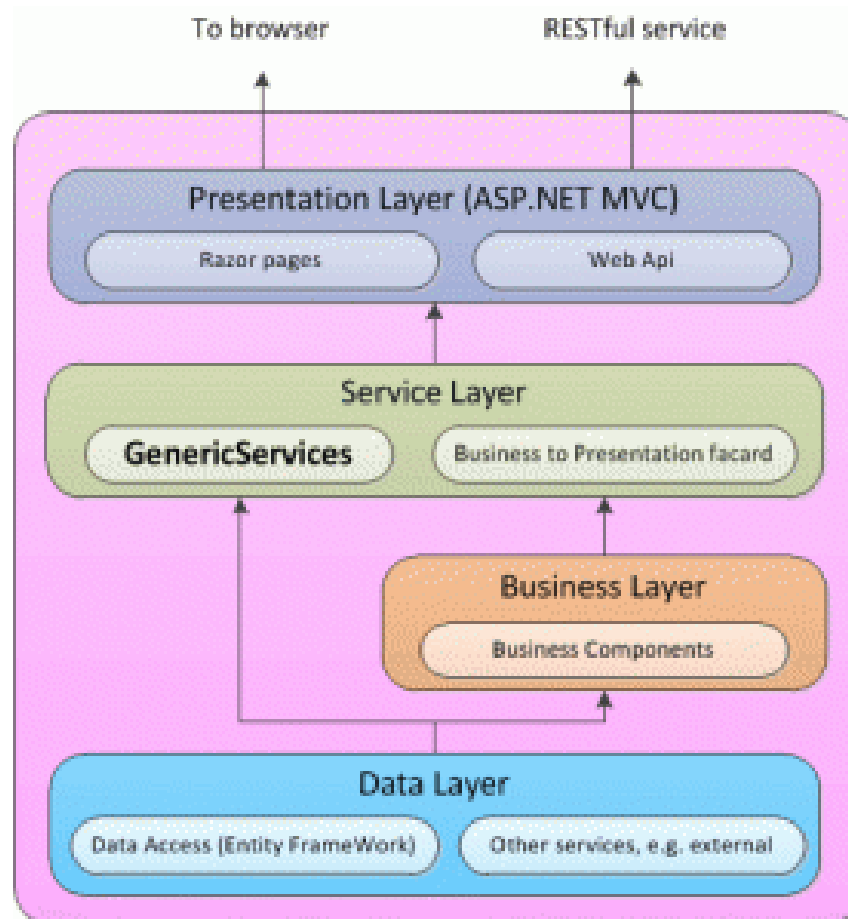
Componentes .NET



Componentes .NET



Componentes .NET Web API



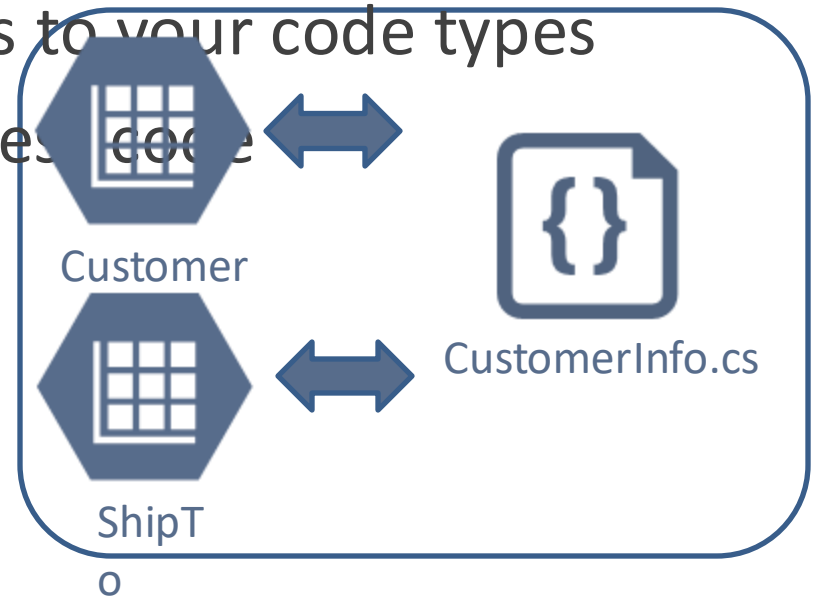
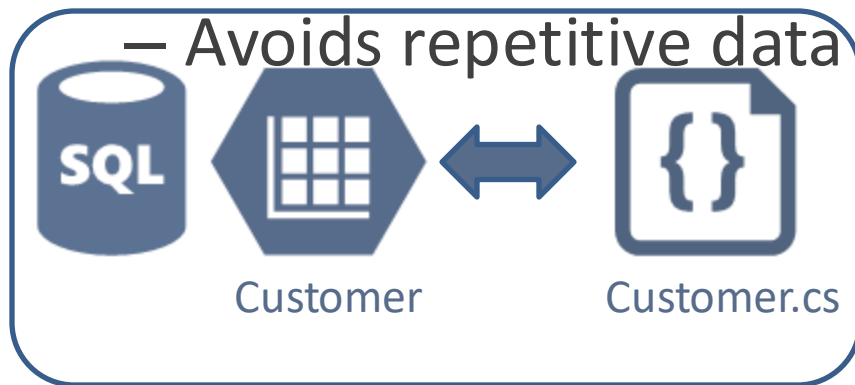
Software Architecture

EntityFramework

What is EntityFramework

- It IS an ORM
- What's an ORM?
 - Maps your database types to your code types

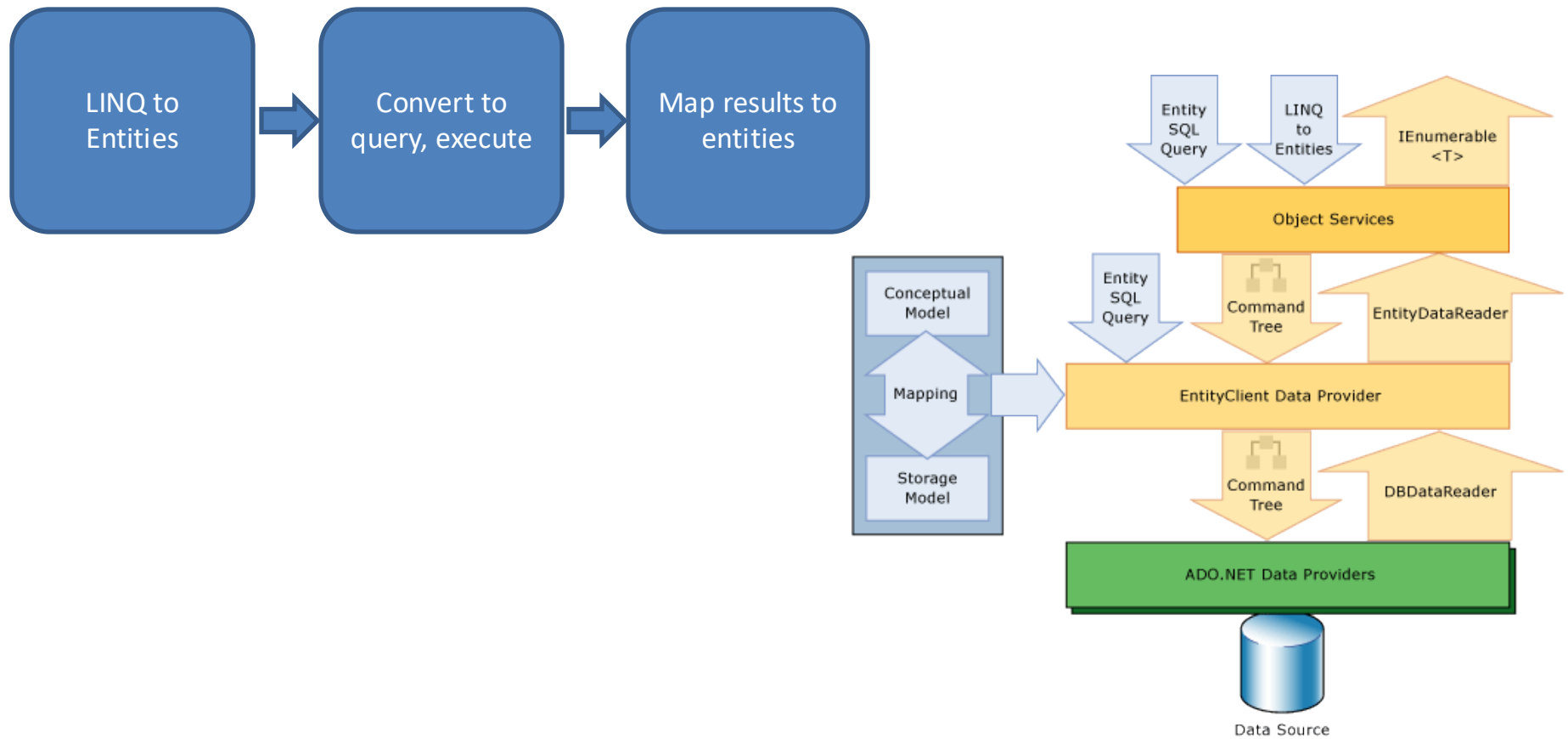
- Avoids repetitive data access code



Some supported features

- Full ORM to map objects to code
- Async Queries
- Stored Procedure mapping
- Reverse engineering existing database
- Using code to create database (code first)
- Concurrency detection

High level view



Code First

- Allows you to use POCO classes as entities
- Uses at a minimum a DbContext and an entity (ie a class)

```
public class Album
{
    public string Name { get; set; }
    public decimal Cost { get; set; }
}

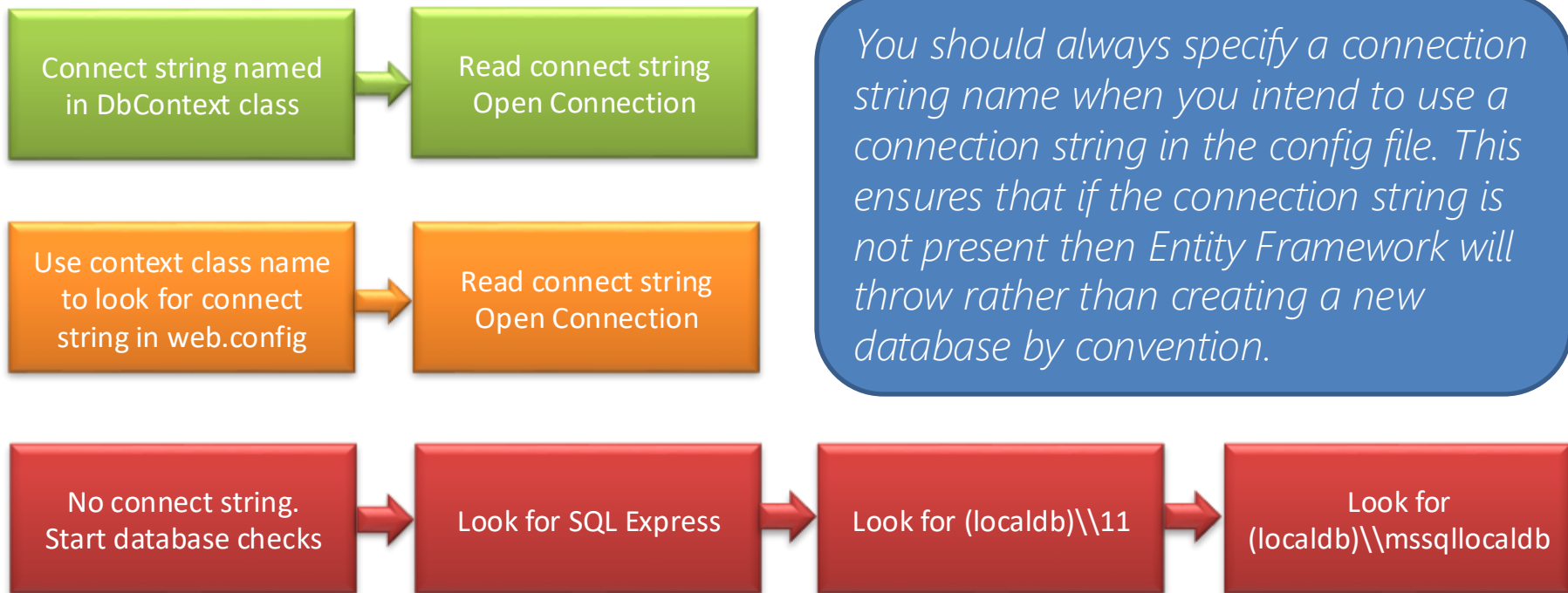
public class MusicStoreDbContext : DbContext
{
    public DbSet<Album> Albums { get; set; }
}
```

How does EF connect to your db?

- Could be cloud, App_Data folder, Intranet, Internet, other folder

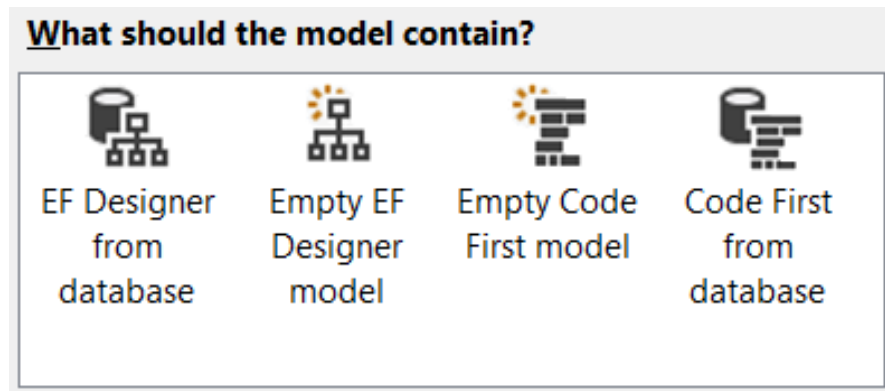
```
public MusicContext() : base("MusicStoreConnection")  
{  
}
```

How does EF connect to your db?



Generating EF classes Ways to generate

- Hand coding classes
- Use a visual designer
- Add->ADO.NET Entity Data Model
- Database First Entity Designer



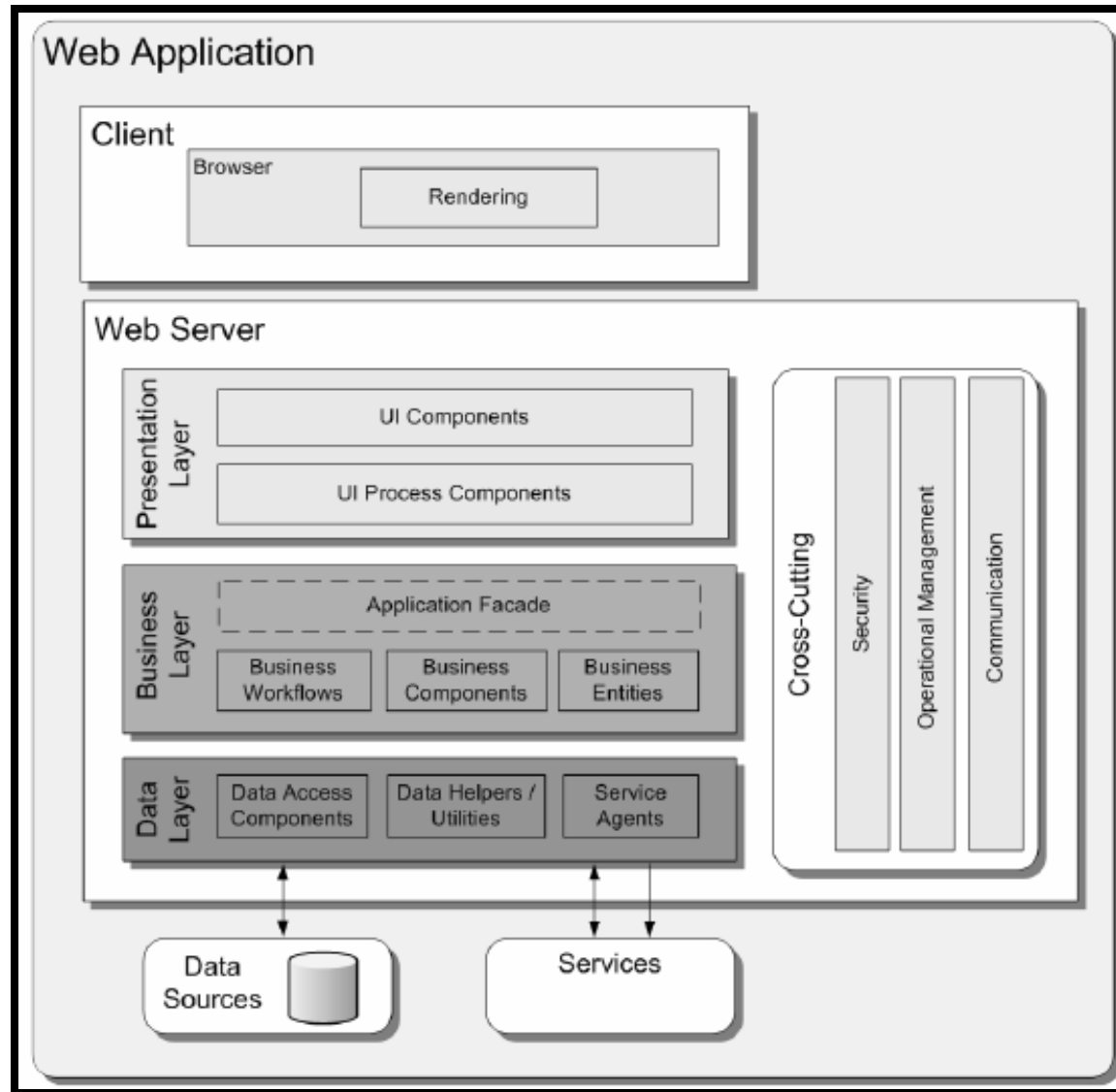
Resources

- EF Resources
 - <https://msdn.microsoft.com/en-us/data/ef.aspx>
- EF Team Blog
 - <http://blogs.msdn.com/b/adonet/>

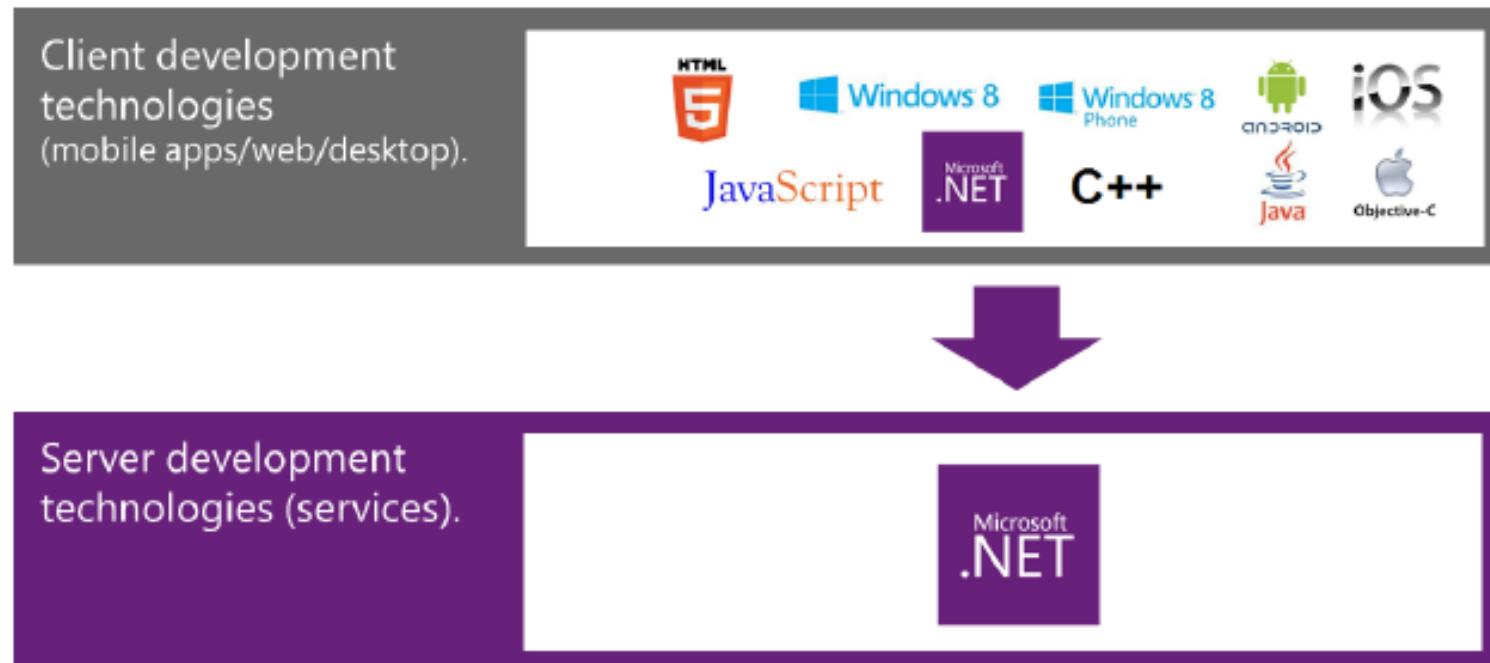
Arquitectura de Software

ASP.NET

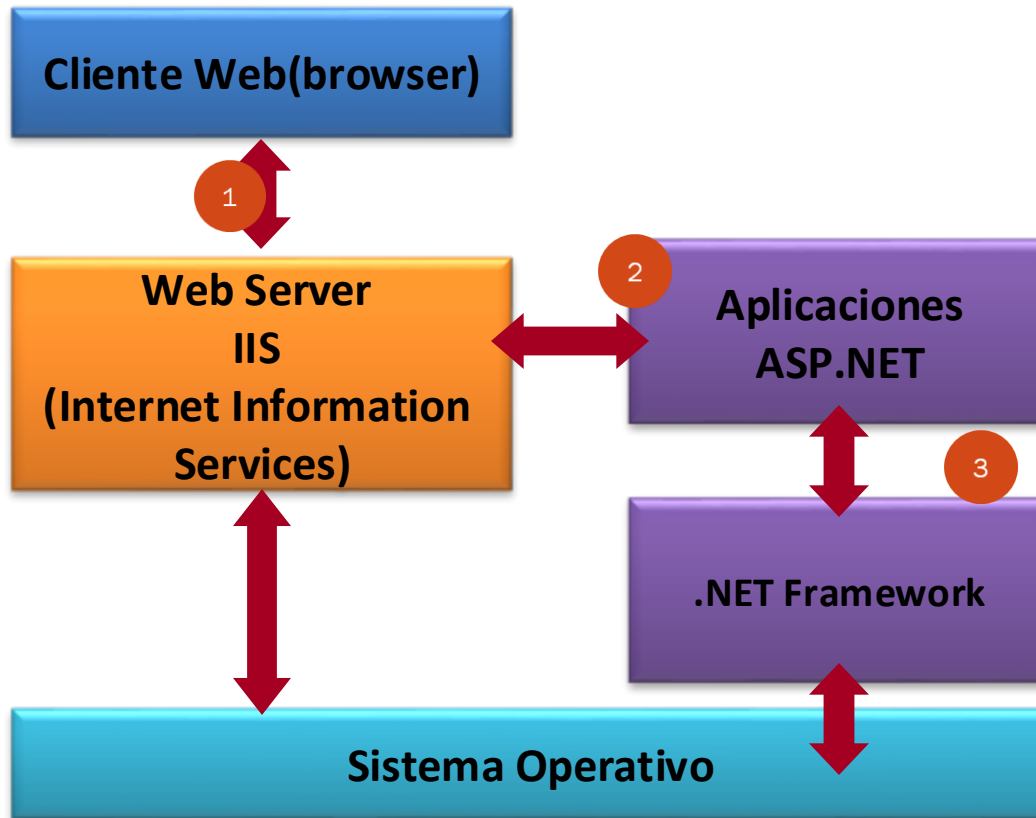
Aplicación Web Típica



Lado del Cliente es independiente del servidor

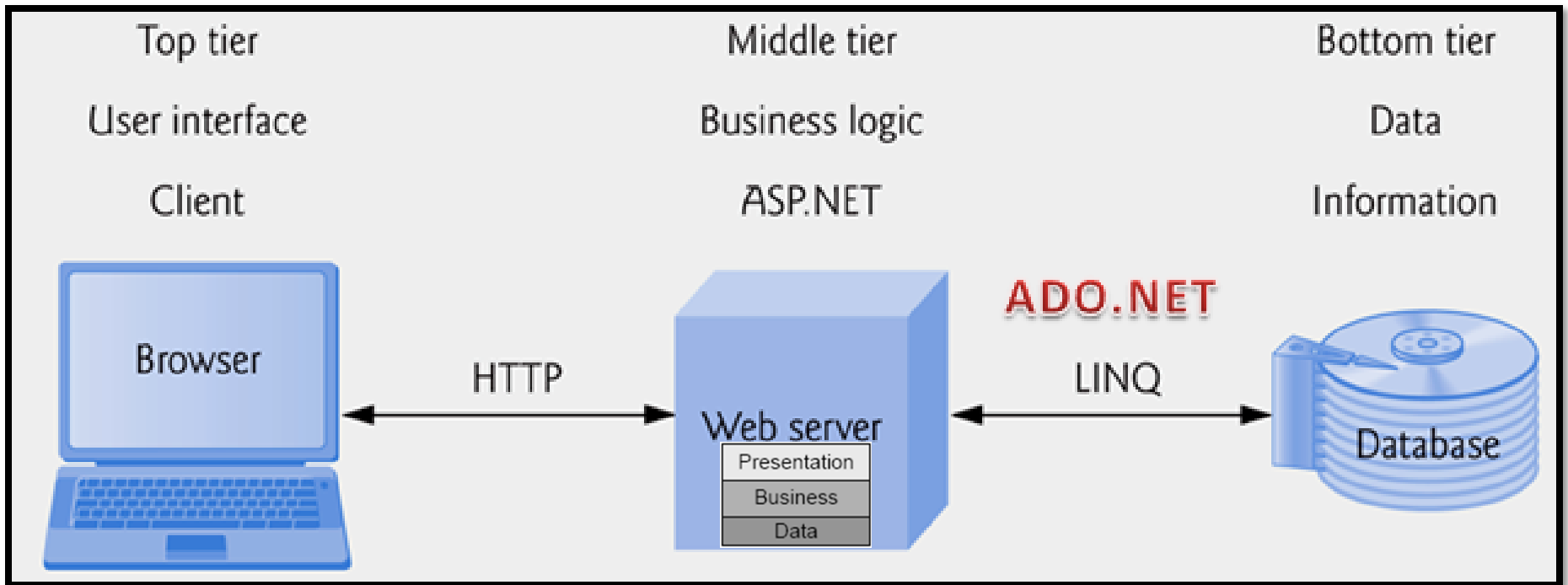


Plataforma de ejecución Web Windows



ASP
(Active Server Pages)

Escenario Web NO Distribuido



- En el middle tier la aplicación web se estructura en 3 capas:
 - Presentación: lógica de presentación
 - Lógica de negocio
 - Acceso a Datos

Capa Presentación Web

- La plataforma provee varias alternativas para construir aplicaciones web
 - ASP.NET MVC (Modelo Vista Controlador)
 - Single Pages (SPA)
 - Web Pages
 - Web Forms
 - No es MVC
 - Forma antigua de hacer páginas web; ya no es muy usada.

Web Forms con soporte HTML

ASP.NET Web Forms with HTML5 support

- For developers who are familiar with Web Forms, this approach provides an easy way to start development while maintaining full control over the code.
- Use libraries like Modernizr for detecting HTML5 features support and workarounds.
- Use CSS3 for less script and more maintainable code.
- Use JavaScript and jQuery for client-side programming.

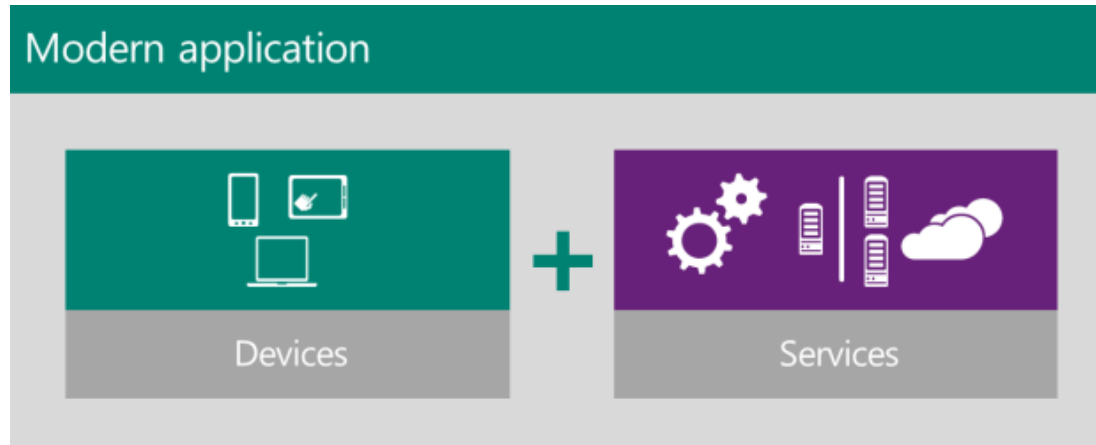
ASP.NET Web Pages

- Es la Tecnología estándar para construir aplicaciones web en la parte de interfaz gráfica de usuario
 - Es una tecnología de lado servidor y no requiere componentes adicionales en el cliente

ASP.NET Web Pages	ASP.NET Web Pages ▪ ASP.NET Web Pages and the Razor syntax provide a fast, approachable, and lightweight way to combine server code with HTML to create dynamic web content.
------------------------------	---

SPA

- Single Page Application



ASP.NET SPA

Single Page Application (SPA) based on **ASP.NET MVC, HTML5/JavaScript, Knockout**, and **Breeze**

- SPA is not a framework but a design pattern that can be implemented by using ASP.NET MVC and heavy use of JavaScript libraries like *Knockout* (for supporting the MVVM pattern within JavaScript) and like the *Breeze* JavaScript library for advanced data management and JavaScript frameworks like *Durandal*.
- Use it for highly interactive web applications and smooth UI updates (minimum server page reload and visible round trips) and when including significant client-side interactions using HTML5, CSS3, and JavaScript.
- Take advantage of the open-source *SignalR* JavaScript client library for bi-directional communication between the client and server.
- Consume ASP.NET Web API services from JavaScript.

ASP.NET MVC

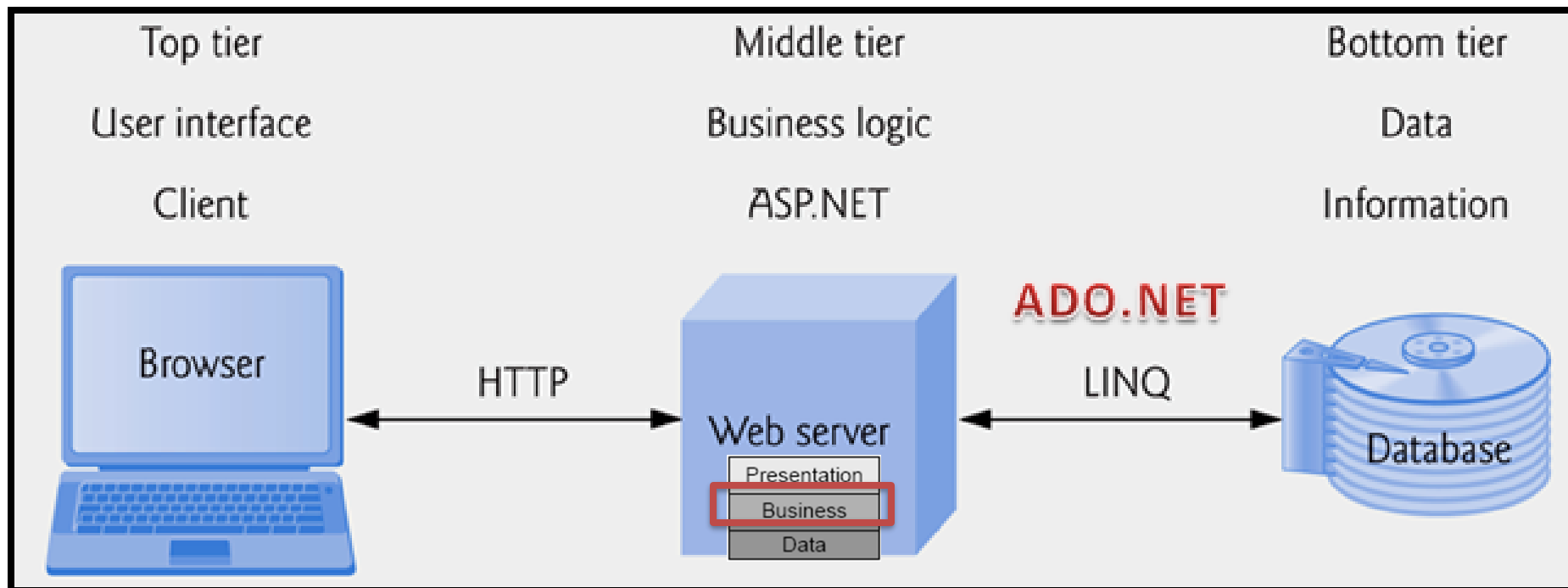
- ASP.NET MVC (Modelo Vista Controlador)
 - Estas aplicaciones se caracterizan por la separación de concerns entre el procesamiento y el rendering de la interfaz de usuario
 - Soporta el TDD (Test Driven Development)

ASP.NET MVC with HTML5 support

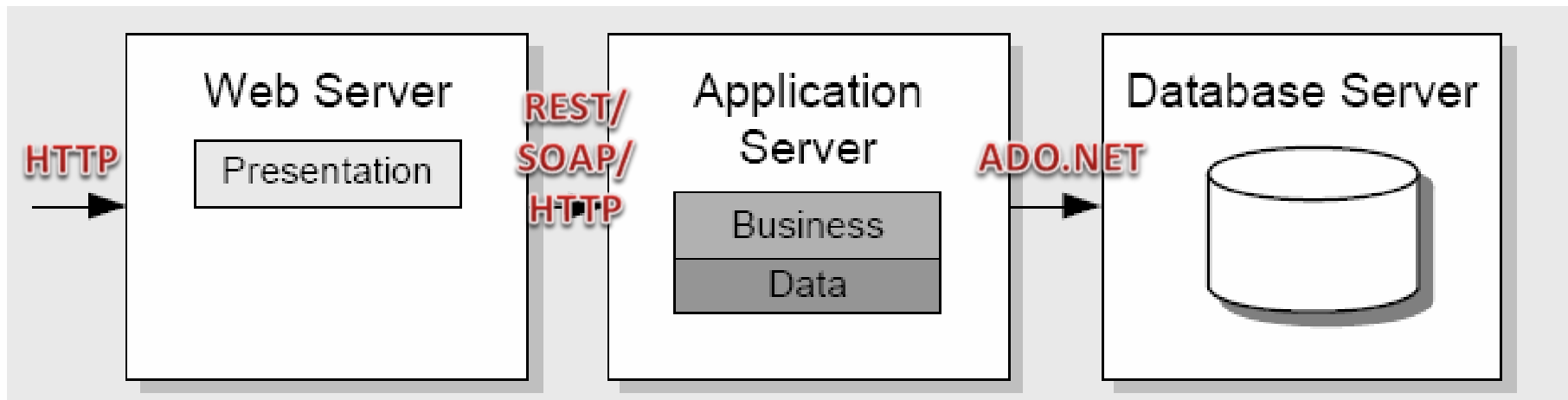
ASP.NET MVC with HTML5 support

- Use for flexible modern and mobile web apps, cross-browser compatibility, mobile web clients that can run on any modern device, and to take advantage of the ASP.NET MVC mobile features (such as using different pages and rendering based on detection of the current browser user agent).
- Offers a full HTML rendering control, better unit testing support, and faking capabilities.
- HTML5: Use libraries like *Modernizr* for detecting HTML5 features support and workarounds.
- Use CSS3 for less script and more maintainable code.
- Use JavaScript and jQuery for client-side programming.
- MVC allows Search Engine Optimizations (SEO) customizations.
- RESTful URLs are an integral part of MVC.

Escenario Web NO Distribuido



Web Distribuida



Software Architecture

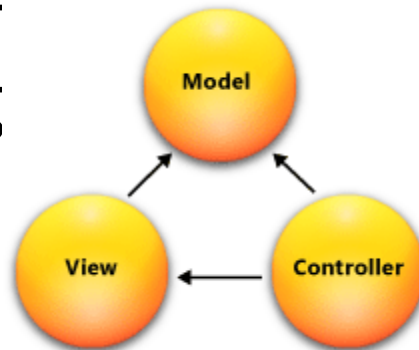
.NET MVC 5.0

MVC

- ASP.NET MVC was designed to facilitate patterns-based development practices such as test-driven development, separation of concerns, inversion of control, and dependency injection.
- Support for Dependency Injection (DI)
 - Class doesn't need to create objects itself
- Support for Inversion of Control (IOC)
 - If an object requires another object, the first should get the second from an outside source (configuration file)

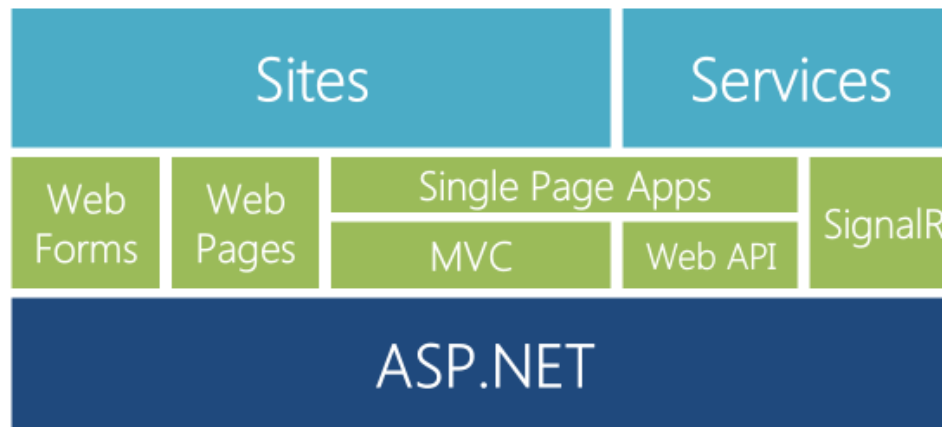
MVC

- The framework encourages separating the business logic layer of a web application from its presentation layer. By dividing the application into models (M), views (V), and controllers (C), *ASP.NET MVC* can make it easier to manage in larger applications.



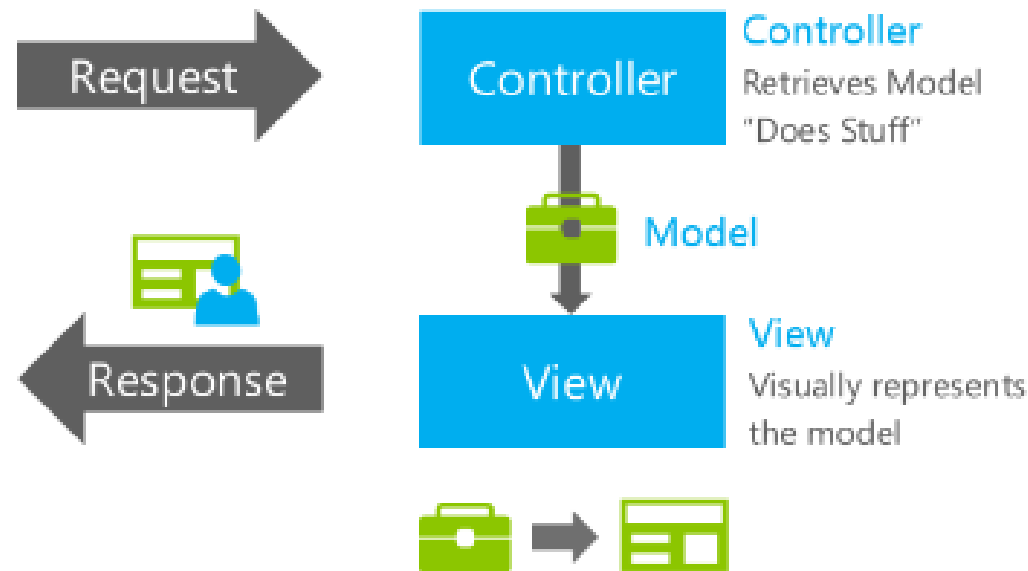
Architecture

ASP.NET Overview



Models, Views, and Controllers

What does MVC look like?



Models, Views, and Controllers

- Models
 - Business/domain logic
 - Model objects, retrieve and store model state in a persistent storage (database).
- Views
 - Display application's UI
 - UI created from the model data
- Controllers
 - Handle user input and interaction
 - Work with model
 - Select a view for rendering UI

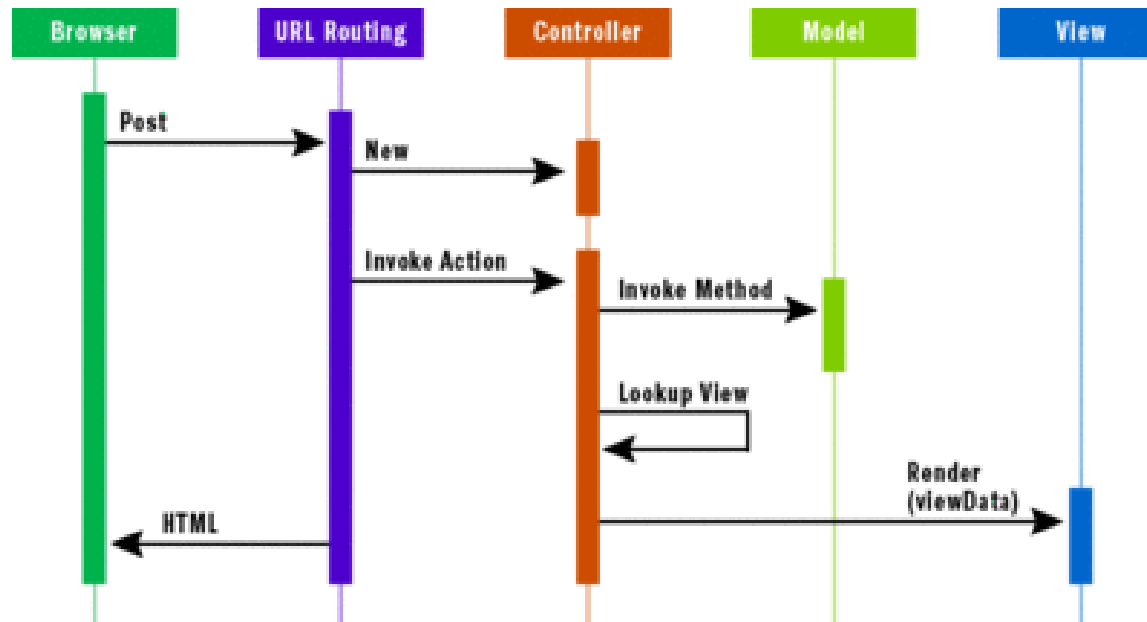
When to use MVC approach?

- Advantages:
 - Easier to manage complexity (divide and conquer)
 - It does not use server forms and view state
 - Front Controller pattern (rich routing)
 - Better support for test-driven development
 - Ideal for distributed and large teams
 - High degree of control over the application behavior

ASP .NET MVC App Structure

- URLs mapped to controller classes
- Controller
 - handles requests,
 - executes appropriate logic and
 - calls a View to generate HTML response
- URL routing
 - ASP .NET routing engine (flexible mapping)
 - Support for defining customized routing rules
 - Automatic passing/parsing of parameters

MVC App Execution



Creating Models

Who Uses the Model?

- User
 - What data will they be interacting with?
- MVC
 - Input controls
 - Display formatting
 - Validation

Preparing Models

- Attributes
 - “Decorate” properties
- Available Attributes
 - DataTypeAttribute
 - DisplayAttribute
 - Validation
 - RequiredAttribute
 - StringLengthAttribute
 - RegularExpressionAttribute
 - CompareAttribute

Data Types

- Validation
- Input controls
 - HTML5
- Available Data Types
 - CreditCard
 - Currency
 - EmailAddress
 - Password
 - Url

How Models Make Life Easier

- Automatic UI creation
- Model Binding
 - Controllers are classes
 - Actions are methods
 - Methods take parameters
 - MVC will convert forms to parameters

Scaffolding

- Generates controllers and views based on models
- Built with common ASP.NET Scaffolding system

Entity Framework

Entity Framework (EF) is an object-relational mapper that enables .NET developers to work with relational data using domain-specific objects. It eliminates the need for most of the data-access code that developers usually need to write.

Controllers

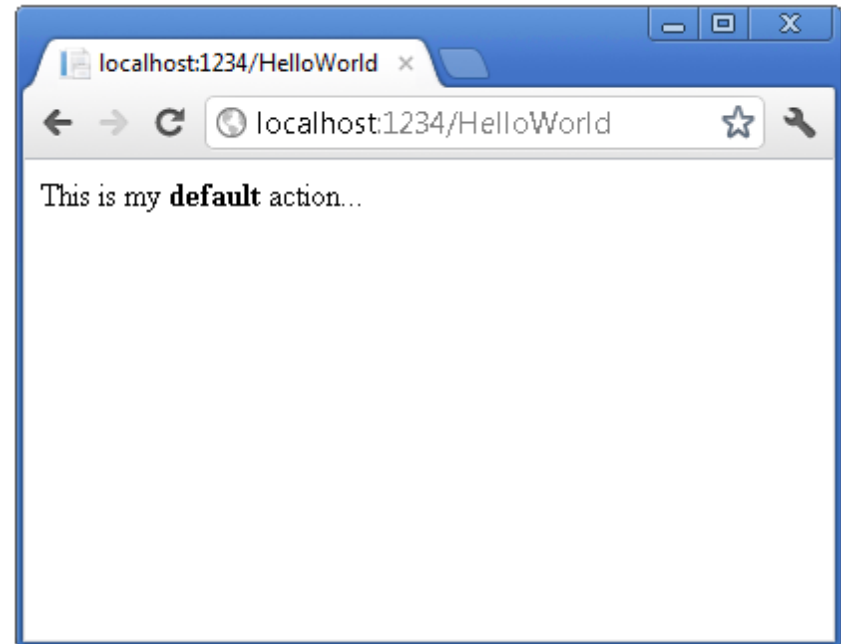
```
using System.Web;
using System.Web.Mvc;

namespace MvcMovie.Controllers
{
    public class HelloWorldController : Controller
    {
        //
        // GET: /HelloWorld/

        public string Index()
        {
            return "This is my <b>default</b> action...";
        }

        //
        // GET: /HelloWorld/Welcome/

        public string Welcome()
        {
            return "This is the Welcome action method...";
        }
    }
}
```



Taking Control of Controllers

- Adding Actions
- Model Binding
- Filters
- Controller Best Practices

Adding Actions

- Controllers are classes
- Actions are methods
- Creating an action involves adding a method to a class

Action Signature

- Return Types
 - ActionResult
 - FileResult
 - JsonResult
 - ViewResult
- Parameters
 - Normal parameters
 - MVC model binding

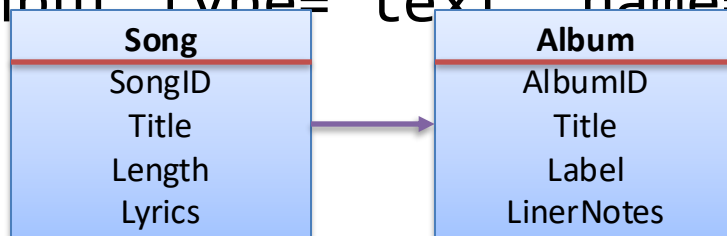
Get and Post

- Create/Update/Delete are typically two step operations
 1. Present the form
 2. Accept the input
- Create two actions
 1. Form presentation via `HttpGet` (default)
 2. Accept data via `HttpPost`

Default Model Binder

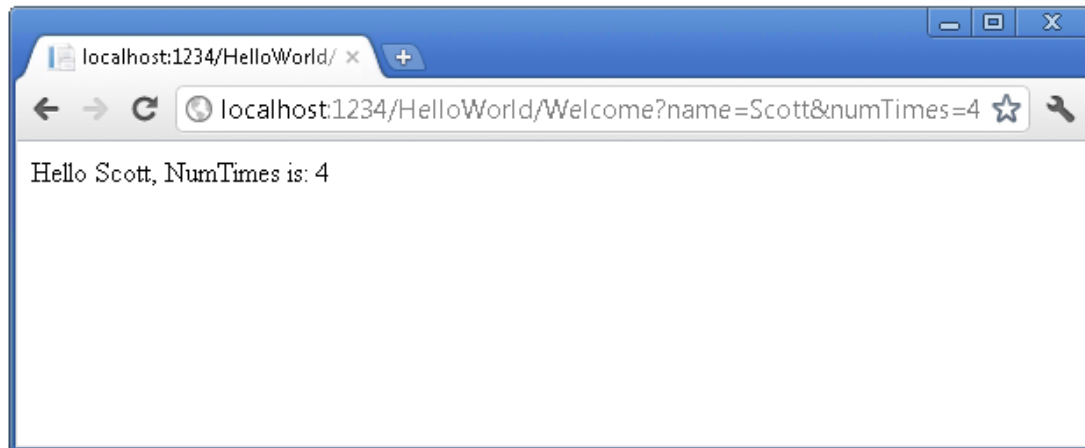
- Uses the name attribute of input elements
 - Automatically matches parameter names for simple data types
 - Complex objects are mapped by property name
 - Complex properties use dotted notation

```
<input type="text" name="Album.LinerNotes" />
```



Parameters

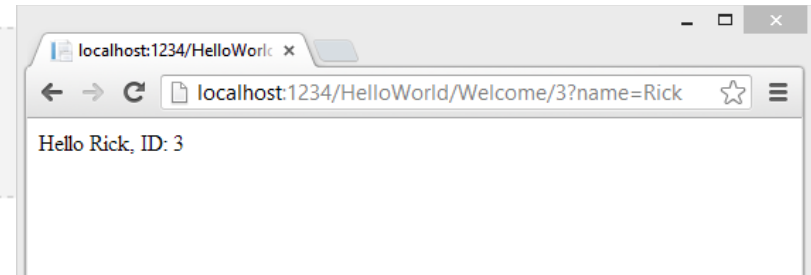
```
public string Welcome(string name, int numTimes = 1) {  
    return HttpUtility.HtmlEncode("Hello " + name + ", NumTimes is: " + numTimes);  
}
```



Parameters

- <http://webapp/HelloWorld/Welcome/3?name=Rick>

```
public string Welcome(string name, int ID = 1)
{
    return HttpUtility.HtmlEncode("Hello " + name + ", ID: " + ID);
}
```



```
public static void RegisterRoutes(RouteCollection routes)
{
    routes.IgnoreRoute("{resource}.axd/{*pathInfo}");

    routes.MapRoute(
        name: "Default",
        url: "{controller}/{action}/{id}",
        defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
    );
}
```

Attribute Routing

- Attributes control routing/URL
- RouteAttribute

```
[Route("Album/Edit/{id:int}")]
```

```
public ActionResult Edit(int id)
```

- www.mymusicstore.com/Album/Edit/42
- Calls the Edit action
- Passes in the ID parameter
- ID must be an integer

RoutePrefix

- Added to controller
- Adds prefix to all routes

```
[RoutePrefix("Album")]  
public class AlbumsController : Controller  
{  
    [Route("Album/Edit/{id:int}")]  
    public ActionResult Edit(int id)  
    {  
        // code  
    }  
}
```

Views

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
}

@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>

<p>Hello from our View Template!</p>
```

Views

- Views created using Razor view engine
- Controller method returns View object
- Controller method return type is ActionResult
- Common pattern: all view pages share the same

```
n public ActionResult Index()  
{  
    return View();  
}
```

Views

Selected master page.

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
}

@{
    ViewBag.Title = "Index";
}

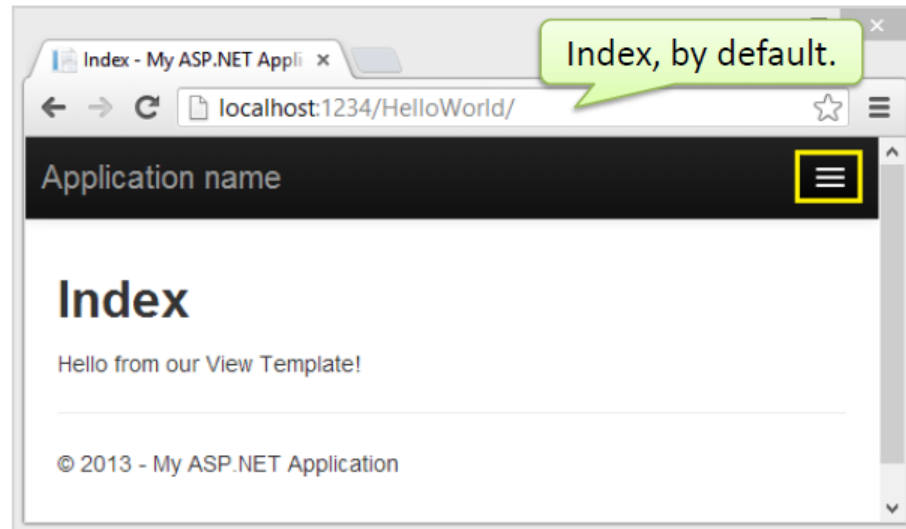
<h2>Index</h2>

<p>Hello from our View Template!</p>
```

```
public ActionResult Index()
{
    return View();
}
```

Change controller's method signature.
The method returns a view object:
searches a view file that is named the
same as the method (Index.cshtml).

Index, by default.



Finding Views

- Views reside in the Views folder
- Subfolders
 - Name of the controller
 - Shared

View Resolution in Action

```
public class AlbumController : Controller
{
    public ActionResult Index()
    {
        return View();
    }
}
```

1. Look for a view named Index in a folder named Album
2. Look for a view named Index in a folder named Shared
3. Error

View Resolution in Action Again

```
public class AlbumController : Controller
{
    public ActionResult Index()
    {
        return View("Default");
    }
}
```

1. Look for a view named Default in a folder named Album
2. Look for a view named Default in a folder named Shared
3. Error

ViewBag

- Pass data between view template and layout view file
- ViewBag is a dynamic object(has no defined properties)

Layout view file.

```
@{
    ViewBag.Title = "Movie List";
}

<h2>My Movie List</h2>

<p>Hello from our View Template!</p>
```

View template file.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@ViewBag.Title - Movie App</title>
    @Styles.Render("~/Content/css")
    @Scripts.Render("~/bundles/modernizr")
</head>
```

Passing data from Controller to View

- View is used for data presentation
- Controller must provide a view with the data
- One approach: using ViewBag
 - Controller puts data to ViewBag,
 - View reads ViewBag and renders the data
 - No data binding!
- Alternative approach: the view model
 - Strongly typed approach

Passing data from Controller to View

Controller

```
using System.Web;
using System.Web.Mvc;

namespace MvcMovie.Controllers
{
    public class HelloWorldController : Controller
    {
        public ActionResult Index()
        {
            return View();
        }

        public ActionResult Welcome(string name, int numTimes = 1)
        {
            ViewBag.Message = "Hello " + name;
            ViewBag.NumTimes = numTimes;

            return View();
        }
    }
}
```

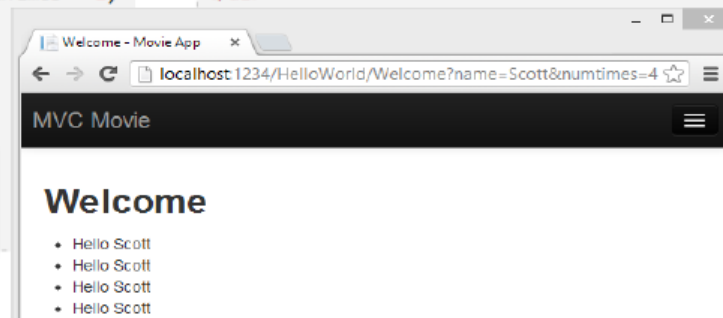
Returns
HelloWorldView
object.

View

```
@{
    ViewBag.Title = "Welcome";
}

<h2>Welcome</h2>

<ul>
    @for (int i = 0; i < ViewBag.NumTimes; i++)
    {
        <li>@ViewBag.Message</li>
    }
</ul>
```



Accessing Model from a Controller

```
using System;
using System.Data.Entity;

namespace MvcMovie.Models
{
    public class Movie
    {
        public int ID { get; set; }
        public string Title { get; set; }
        public DateTime ReleaseDate { get; set; }
        public string Genre { get; set; }
        public decimal Price { get; set; }
    }

    public class MovieDbContext : DbContext
    {
        public DbSet<Movie> Movies { get; set; }
    }
}
```

EF namespace
DbContext
DbSet

EF database
context
FETCH,
INSERT,
UPDATE

Accessing Model from a Controller

```
public class MoviesController : Controller
{
    private MovieDbContext db = new MovieDbContext();

    // GET: /Movies/
    public ActionResult Index()
    {
        return View(db.Movies.ToList());
    }
}
```

Instantiated
DbContext instance.

Index method.

Views and Models

- Views have a `Model` property
- Type is set by `@model` declaration
- Advanced note
 - Views do not need to have a typed model
 - Useful for creating dynamic views

Razor Syntax

- @ indicates server-side code

@model: Specifies class of the model

Id parameter generally passed as a part of the route.

```
public ActionResult Details(int? id)
{
    if (id == null)
    {
        return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    }
    Movie movie = db.Movies.Find(id);
    if (movie == null)
    {
        return HttpNotFound();
    }
    return View(movie);
}
```

@model MvcMovie.Models.Movie

```
@{
    ViewBag.Title = "Details";
}
```

Communicates with the master page.

<h2>Details</h2>

<div>

<h4>Movie</h4>

<hr />

<dl class="dl-horizontal">

<dt>

@Html.DisplayNameFor(model => model.Title)

</dt>

@*Markup omitted for clarity.*@

</dl>

</div>

<p>

@Html.ActionLink("Edit", "Edit", new { id = Model.ID }) |

@Html.ActionLink("Back to List", "Index")

</p>

Context-sensitive data access.

HtmlHelper

- Helps generate HTML
- Uses attributes on model
 - Display names
 - Formatting
 - Input elements

HtmlHelper and Lambdas

`@Html.DisplayFor(model => model.Name)`

- MVC uses reflection
- Helper methods need the property, not the value
 - `@Html.DisplayFor(Model.Name)` would pass the value of Name
- Consider `@Html.DisplayFor(m => m.Name)` for easier reading

Displaying Data

- **DisplayNameFor**
 - DisplayName attribute
 - Display attribute, Name property
- **DisplayFor**
 - Uses DisplayFormat (if applicable)

Creating Forms

- `HtmlHelper.BeginForm()`
- Why not just use a form element?
 - URLs can always change
- Parameters
 - Action name
 - Controller name
 - Form method
 - Get
 - Post

Accepting Input

- **LabelFor**
 - Creates a label element
 - Useful for touch
- **InputFor**
 - Creates input element
 - Uses HTML5 based on DataType attribute

Validation

- **ValidationMessageFor**
 - Display error message next to text box
- **ValidationSummary**
 - Display all error messages in one location

Referencias

- <https://www.asp.net/mvc/overview/getting-started/introduction/getting-started>
- https://mva.microsoft.com/en-us/training-courses/introduction-to-asp-net-mvc-8322?l=nKZwZ8Zy_3504984382
- Microsoft Virtual Academy. Jon Galloway | Technical Evangelist. Christopher Harrison | Content Developer.

Bibliografía

- Deitel & Deitel. Programación en la web. 2008. Pearson Education.
- Application Architecture Guide 2.0. *Designing Applications on the .NET Platform. Microsoft. 2009.*
- Conferencias técnicas MSDN

Bibliografía

- Deitel & Deitel. Programación en la web. 2008. Pearson Education.
- Application Architecture Guide 2.0. *Designing Applications on the .NET Platform*. Microsoft. 2009.
- <https://www.campusmvp.es/recursos/post/El-panorama-de-NET-en-2016-plataforma-completa-o-NET-Core.aspx>
- <https://msdn.microsoft.com/es-es/library/hh425099.aspx>